

PROJECT REPORT

Student Performance Management System

A Beginner-Friendly Console-Based Java Application

Academic Project Report

Project: Student Performance Management System

Technology: Java

Application Type: Console-Based

Data Storage: Local Text File

Prepared as an academic Java project

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Scope of the Project
5. System Requirements
6. Technologies Used
7. System Architecture
8. Functional Modules
9. Data Management
10. Input Validation and Error Handling
11. Implementation and Working
12. Testing
13. Results and Discussion
14. Limitations
15. Future Enhancements
16. Conclusion
17. References

1. Introduction

The Student Performance Management System is a console-based Java application created to manage student records, academic marks, attendance, and performance reports. The project focuses on core Java programming rather than external frameworks or a database. It is intended to demonstrate practical use of classes and objects, collections, file handling, exception handling, validation, and menu-driven interaction.

The application keeps student information in a local text file. This allows records to remain available after the program is closed and started again. The documented feature set covers student management, marks management, attendance tracking, individual reports, and a class performance dashboard.

1.1 Project Overview

The system is operated through a terminal. A user selects an option from the main menu and the corresponding manager class performs the requested operation. The Student class represents the student data, manager classes handle major operations, FileManager handles file persistence, and InputValidator handles validation of user input.

1.2 Purpose of the Project

The purpose of the project is to provide a simple academic example of how a small Java application can be divided into understandable classes while still supporting useful operations such as CRUD-style student management, marks calculations, attendance calculations, reporting, and persistent local storage.

2. Problem Statement

Student details, marks, and attendance are often maintained separately. For a small academic environment, this can make it difficult to search records, update information, calculate performance, and view a summary of a class. The project addresses this problem with one menu-driven console application that brings these functions together in a simple format.

The system is intentionally small and does not attempt to solve institutional-scale student administration. Instead, it demonstrates a clear and understandable implementation suitable for learning and academic evaluation.

3. Objectives

- Create a simple Java application for managing student records.
- Implement add, view, search, update, and delete operations for student data.
- Store marks for Java, Mathematics, and English and calculate total, average, and grade.
- Record attendance and calculate attendance percentage and eligibility.
- Generate an individual performance report and a class performance dashboard.
- Use local file handling so data persists between program runs.
- Apply input validation and exception handling to reduce invalid input and prevent crashes.
- Demonstrate practical use of Java OOP and the Collections Framework.

4. Scope of the Project

The project covers the core academic operations documented in the README. It is a local, single-application console program using a text file for data storage.

Area	Scope
Student records	Add, view, search, update, and delete student

	information.
Marks	Enter subject marks, calculate total and average, assign grade, validate range.
Attendance	Record classes, calculate attendance percentage, check eligibility.
Reports	Generate individual student performance information.
Dashboard	Summarize class average, highest-performing student, attendance statistics, and performance categories.
Persistence	Load and save records using students.txt.
Interface	Terminal/console menu.

The documented future scope includes a graphical user interface, database integration, authentication, more subjects, PDF report export, monthly attendance tracking, and more advanced analytics.

5. System Requirements

The project uses standard Java features and does not require external libraries according to the project README.

5.1 Software Requirements

- Java Development Kit (JDK).
- A terminal or command prompt.
- Visual Studio Code or another Java-supported IDE (optional).
- Git for obtaining the repository (optional if the source is already available locally).

5.2 Hardware Requirements

- A computer capable of running a standard JDK and terminal application.
- Sufficient storage for the Java source files and local text data.

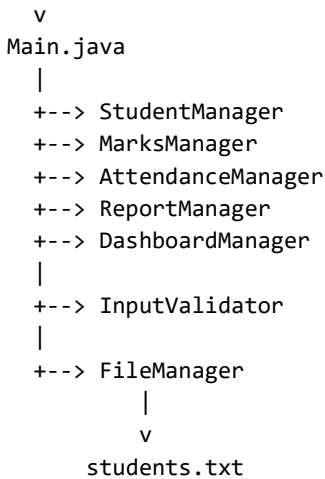
6. Technologies Used

Technology / Concept	Use in the Project
Java	Primary programming language.
Object-Oriented Programming	Used to organize student data and application responsibilities into classes.
ArrayList / Collections Framework	Used for in-memory collection management.
File Handling	Used to store and reload student information from students.txt.
Exception Handling	Used to handle invalid user input and file-related situations.
Git and GitHub	Used for source-code versioning and repository management.
Visual Studio Code / Java IDE	Development environment support.

7. System Architecture

The project follows a simple class-based structure rather than a large multi-layer enterprise architecture. The aim is to keep the application understandable for a beginner while separating major responsibilities.

User
|



7.1 Main Components

File	Responsibility
Main.java	Application entry point and terminal menu flow.
Student.java	Represents student information and related data.
StudentManager.java	Handles student record operations such as add, view, search, update, and delete.
MarksManager.java	Handles marks entry and performance calculations.
AttendanceManager.java	Handles attendance entry, calculation, and eligibility checks.
ReportManager.java	Generates individual student performance reports.
DashboardManager.java	Produces class-level performance summaries.
FileManager.java	Handles reading and writing persistent data.
InputValidator.java	Checks user input and prevents invalid values.

8. Functional Modules

8.1 Student Management

The Student Management module provides the basic record lifecycle. It supports adding students, viewing existing students, searching by roll number, updating student information, and deleting records. Duplicate roll numbers are prevented according to the documented requirements.

8.2 Marks Management

The Marks Management module records marks for Java, Mathematics, and English. It calculates total marks, average marks, and grade. Marks are validated so that values remain within the range of 0 to 100.

8.3 Attendance Management

The Attendance Management module records the total number of classes and attended classes. It calculates attendance percentage and determines attendance eligibility. Invalid attendance values are rejected.

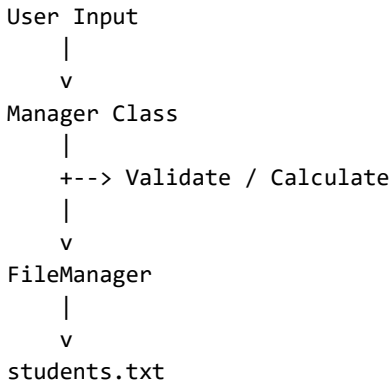
8.4 Reports and Dashboard

The Report module generates an individual performance report. The Dashboard module provides class-level information such as class average, highest-performing student, attendance statistics, and performance categories.

9. Data Management

The application uses `students.txt` as its local data store. The file-based approach keeps the project simple and avoids the need for database configuration. When the program starts, saved records can be loaded from the file, and updates can be written back so information is available after a restart.

9.1 Data Flow



9.2 Advantages of the Chosen Storage Method

- Easy to understand for beginners.
- No database server or connection configuration is required.
- Data remains available after restarting the program.
- Keeps the academic project focused on core Java programming.

10. Input Validation and Error Handling

The project includes an `InputValidator` class and documents validation rules for common invalid cases. The goal is to keep the console application stable and make errors understandable to the user.

Validation Rule	Expected Behavior
Numeric fields	Reject non-numeric input where a number is required.
Marks	Accept only values from 0 to 100.
Attendance	Reject negative values and values where attended classes exceed classes held.
Student name	Reject empty names.
Roll number	Prevent duplicate roll numbers.

Exception handling is used alongside validation so that invalid input or file-related problems can be handled without causing the entire program to terminate unexpectedly.

11. Implementation and Working

11.1 Program Start

The application starts from `Main.java`. The terminal menu is presented and the user selects an operation by entering the corresponding option number.

11.2 Menu Operations

1. Add Student

2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Add Marks
7. View Performance
8. Record Attendance
9. View Attendance
10. Generate Performance Report
11. Class Performance Dashboard
12. Exit

11.3 Terminal Deployment and Execution

The project is designed to be compiled and run from a terminal. With the simplified repository structure, Java source files are stored inside src while students.txt remains at the project root. Compiled class files are placed in an out directory so generated .class files do not mix with source code.

```
git clone https://github.com/vaibhavshuklajune23-ctrl/Student-Performance-Management-System.git
cd Student-Performance-Management-System
mkdir out
```

11.4 Windows

```
javac -d out src\*.java
java -cp out Main
```

11.5 macOS / Linux

```
javac -d out src/*.java
java -cp out Main
```

Running java -cp out Main from the project root keeps the program in the correct working directory so the root-level students.txt file can be used for persistence.

12. Testing

The project documentation identifies testing areas covering the major functions of the application. Testing is intended to verify normal operations, validation rules, calculations, reporting, dashboard behavior, persistence, and invalid input handling.

Test Area	Purpose
Student creation	Verify that a student record can be added.
Duplicate roll number	Verify duplicate student identifiers are rejected.
Search	Verify student lookup works by roll number.
Update	Verify existing student information can be changed.
Delete	Verify a selected student can be removed.
Marks validation	Verify marks outside 0-100 are rejected.
Attendance validation	Verify invalid attendance values are rejected.
Performance calculations	Verify total, average, and grade calculations are produced.
Reports	Verify individual performance information can be generated.
Dashboard	Verify class-level summaries are displayed.
Data persistence	Verify stored records are available after restart.
Invalid input handling	Verify the program responds safely to invalid input.

13. Results and Discussion

Based on the documented project scope, the system combines the major academic record functions in one console application. The result is a small and understandable Java program with separate classes for student operations, marks, attendance, reports, dashboard summaries, validation, and file persistence.

The project is intentionally focused on functionality and learning outcomes rather than a graphical interface or enterprise infrastructure. The design demonstrates how a menu-driven Java program can use classes, collections, validation, calculations, and file handling together to build a complete academic application.

14. Conclusion

The Student Performance Management System is a beginner-friendly Java console application that demonstrates the practical use of core programming concepts in a single project. It provides student record management, marks management, attendance tracking, performance reporting, class-level summaries, input validation, and local data persistence.

The simplified repository structure supports the same goal: keeping the project easy to understand while maintaining a clear separation between source code, documentation, screenshots, the academic report, and project data. The project provides a practical foundation that can later be extended with a database, graphical interface, authentication, and richer analytics.

15. References

13. Project README: Student Performance Management System.
14. Java Documentation / Java Platform API documentation for core Java features used by the project.